SEC-RUNBOOK-01

SEC-RUNBOOK-01: Local AI Infrastructure & Security Ledger

Status: FINAL **Author:** Cybersecurity Admin (Kelly Guzman) **Classification:** PUBLIC - PORTFOLIO ARTIFACT **Target Environment:** Local Workstation

1. Environment Baseline & Network Configuration

- Host role: Local administrative workstation
- Host OS: Windows (PowerShell environment)
- Model runner: Ollama (Version 0.31.1+) [2026 baseline]
- Listening address: `127.0.0.1:11434` (loopback interface only)

Ollama is a local model runner. It loads open-weight language models and exposes an HTTP API on the loopback interface at `127.0.0.1:11434`. Nothing in this setup trains or builds models; it downloads published weights and serves inference requests locally.

1.1 Cross-Origin Resource Sharing (CORS) Configuration

CORS controls which web-page origins a browser will allow to read responses from the Ollama HTTP API. It is a browser-enforced control, and it is set through the `OLLAMA_ORIGINS` environment variable.

Prior configuration (misconfiguration): The endpoint was previously started with a wildcard origin:

```
$env:OLLAMA_ORIGINS="*"; ollama serve
```

This was documented as a hardening step, but it is the opposite. A wildcard causes Ollama to return CORS headers (`Access-Control-Allow-Origin: *`) permitting any web origin, which the browser then honors by allowing the page to read the API's responses. While `ollama serve` is running, any site loaded in the browser can reach `127.0.0.1:11434`, enumerate the installed models, and issue inference calls. Binding to loopback (`127.0.0.1`) blocks remote hosts only; it does not block a web page running in the local browser.

Current configuration (remediated): The allowed origin is pinned to the single browser extension that legitimately needs access:

```
$env:OLLAMA_ORIGINS="chrome-extension://<EXTENSION_ID>"; ollama serve
```

Pinning a single extension origin limits which additional external browser origins are permitted; it does not make that extension the exclusive allowed origin. `OLLAMA_ORIGINS` does not replace Ollama's allowed-origin set; it appends to it. Ollama always permits a built-in list of default origins that cannot be removed through this variable:

```
http://localhost   https://localhost   http://localhost:*   https://localhost:*
http://127.0.0.1   https://127.0.0.1   http://127.0.0.1:*   https://127.0.0.1:*
http://0.0.0.0     https://0.0.0.0     http://0.0.0.0:*     https://0.0.0.0:*
app://*   file://*   tauri://*   vscode-webview://*   vscode-file://*
```

So pinning the extension narrows which external web origins can reach the API, while every default origin above remains allowed regardless of the variable's value.

Residual risk: Any process running on the host can still reach the loopback socket directly, because Ollama has no authentication layer. CORS constrains browser-originated requests only; it does not authenticate or restrict local processes. Reducing that residual risk requires host-level controls (for example, a local firewall rule or a reverse proxy that adds authentication), not an origins setting.

2. Local Models

The models below are Meta open-weight releases. They are downloaded and run locally via Ollama; no fine-tuning or local

training is performed. Model files are stored under `%USERPROFILE%\.ollama\models`. No external telemetry is enabled.

Model	Parameters	Publisher / Variant	Use Case
Llama 3.1	8B	Meta, instruction-tuned	Private administrative tasks (SBA, medical, financial records)
Llama 3.2	3B / 1B	Meta, lightweight	Fast inline browser tasks / Proton Mail context parsing

3. Local Storage & Retrieval (RAG)

- Data source: Open WebUI local vector store (offline)
- Data handling: Files are split into chunks and embedded locally. No telemetry is sent externally and no fine-tuning pipeline is used; data stays on the local disk.
- Ingested content: Local business operational documents, administrative workflows, and historical session logs.

4. Lab & Sandbox Logs

4.1 Python Reachability Check - 2026-07-30

A small script confirms the local Ollama API is reachable and responds on the `/api/tags` endpoint. This is a reachability check only, not a CORS validation. CORS is enforced by the browser; the Python `requests` library ignores CORS response headers, so the script returns 200 whether or not the request origin is on the allowed list. The `Origin` header is included only to mirror how a browser client would label the request. This sample reflects the remediated configuration: the pinned extension origin shown here was applied on the remediation date (2026-07-30), so the sample is dated to match rather than to the original July 2, 2026 reachability test that used the prior settings.

```
        import requests

        def verify_local_ai_infrastructure():
            url = "http://127.0.0.1:11434/api/tags"
            headers = {"Origin": "chrome-extension://<EXTENSION_ID>"}
            print("[*] Checking local Ollama API...")
            try:
                response = requests.get(url, headers=headers, timeout=5)
                if response.status_code == 200:
                    print("[+] SUCCESS: Local API is reachable.")
                else:
                    print(f"[-] FAILED: Status {response.status_code}")
            except requests.exceptions.ConnectionError:
                print("[-] CRITICAL: Port 11434 unreachable.")

        if __name__ == "__main__":
            verify_local_ai_infrastructure()
```

4.1b CORS Preflight Check - curl

To actually exercise the CORS policy (rather than just reachability), send a preflight OPTIONS request with an origin that should not be allowed and inspect the response headers:

```
        curl -X OPTIONS http://127.0.0.1:11434 \
          -H "Origin: https://example.com" \
          -H "Access-Control-Request-Method: GET" \
          -I
```

The `Access-Control-Allow-Origin` value in the response is what confirms whether the origin is permitted: an allowed origin is echoed back in that header, while a disallowed origin is not.

4.2 Network Topology & Port Binding

```
[Local browser / client] --> [127.0.0.1:11434 loopback] --> [Ollama local runtime]
```

- **CORS binding rule:** The allowed browser origin is pinned to a single extension via `$env:OLLAMA_ORIGINS="chrome-extension://<EXTENSION_ID>"`. A wildcard (`*`) was the prior configuration; that was a misconfiguration, because it allows any visited web page to reach the API. The control is now a single pinned external

origin. `OLLAMA_ORIGINS` appends to Ollama's built-in default origins (`localhost`, `127.0.0.1`, `0.0.0.0`, `app://`, `file://`, `tauri://`, `vscode-webview://`, `vscode-file://`) rather than replacing them, so the pinned extension is the only added external origin, not the sole allowed origin. Loopback binding blocks remote hosts but not the local browser, which is why the origin restriction is required in addition to loopback binding.

- **Data handling:** Model files and ingested data stay on the local disk under `%USERPROFILE%\.ollama\models`. No external telemetry is enabled.
- **Residual risk:** Ollama has no authentication layer, so any local process can still reach the loopback socket regardless of the CORS setting.

5. Findings and Remediation

The following finding was self-identified during review of this runbook and is logged here in the same format used across the NIST 800-53 package.

Finding SEC-RUNBOOK-01-F1: CORS wildcard origin on local model API

Field	Detail
Date identified	2026-07-30
Date remediated	2026-07-30
Status	Closed
Severity	Medium
Affected control(s)	AC-4 (Information Flow Enforcement), SC-7 (Boundary Protection), CM-6 (Configuration Settings)
Component	Ollama HTTP API, 127.0.0.1:11434

Description: The Ollama endpoint was configured with `OLLAMA_ORIGINS="*"` and this wildcard was documented as a hardening control. In fact a wildcard origin causes Ollama to return `Access-Control-Allow-Origin: *`, which the browser then honors by allowing the page to read the API's responses. While `ollama serve` was running, any site loaded in the local browser could reach the API, enumerate installed models, and issue inference calls. Loopback binding did not mitigate this, as it restricts remote hosts only and not pages executing in the local browser.

Remediation: Replaced the wildcard with a single pinned origin, `OLLAMA_ORIGINS="chrome-extension://<EXTENSION_ID>"`, limiting the additional browser-originated cross-origin access to the one extension that requires it. `OLLAMA_ORIGINS` appends to Ollama's non-removable default origins rather than replacing them, so the pinned extension is the only added external origin and not the exclusive allowed origin. Updated Sections 1.1 and 4.2 to describe the control accurately.

Residual risk (accepted): Ollama provides no authentication on the local API, so any process on the host can still reach the loopback socket. This is inherent to the current Ollama design and is out of scope for the CORS control; it would be addressed by host-level controls (local firewall rules or an authenticating reverse proxy) if the threat model requires it. In addition, Ollama's built-in default origins, including `file://` and `app://`, cannot be removed through `OLLAMA_ORIGINS` and are therefore an accepted residual risk of the current design, alongside the no-authentication item above.